

Proyecto Desarrollo de Aplicaciones Multiplataforma

De Jose María de Puelles Gutiérrez



Índice

1. Introducción.....	5
1.1 Justificación del Proyecto.....	5
1.2 Análisis Comparativo de Aplicaciones Similares.....	5
1.3 Tendencias Actuales.....	6
1.4 Beneficios y Expectativas.....	6
2. Descripción del Proyecto.....	7
2.1 Tipo de Proyecto.....	7
2.2 Características Principales.....	7
2.3 Usuarios destinatarios.....	7
3. Objetivos del Proyecto.....	8
3.1 Objetivo General.....	8
3.2 Objetivos Específicos.....	8
4. Alcance del proyecto.....	8
5. Requisitos del Proyecto.....	9
5.1 Requisitos Funcionales.....	9
5.2 Requisitos Técnicos.....	10
5.3 Requisitos Legales o Normativos.....	10
6. Planificación del proyecto.....	10
6.1 Estructura de tareas.....	10
6.2 Cronograma de Gantt.....	11
6.3 Recursos Necesarios.....	11
7. Plan de gestión de riesgos.....	12
7.1 Identificación y evaluación de riesgos.....	12
7.2 Recursos preventivos.....	13
7.3 Plan para mitigar las consecuencias (Contingencia).....	13
8. Diseño.....	14
8.1. Prototipado (Wireframe de la aplicación).....	14
Autenticación y registro (AuthScreen).....	14
Exploración de torneos (HomeScreen).....	15
Gestión de Usuarios (UsersScreen).....	15
Detalle de Perfil (ProfileScreen).....	15
Configuración y Personalización (SettingsScreen).....	16
Gestión del tema.....	16
Idioma (no implementado).....	16
Notificaciones (no implementado).....	16
8.2 Modelo de Datos.....	17
1. Entidad Profile (Usuario).....	17
2. Entidad Tournament (Torneo).....	17
3. Entidad Enrollment (Inscripción).....	18
4. Entidad Review (Reseña).....	18
8.3. Especificaciones Técnicas.....	19
Tecnologías y arquitectura.....	19

Lógica de negocio en servidor.....	19
Habilitación de Políticas.....	22
Políticas de Perfiles.....	22
Políticas de torneos.....	22
Políticas de Inscripciones.....	23
Políticas de Reseñas.....	24
Organización del proyecto y estructura de paquetes.....	24
data/.....	24
ui/.....	25
utils/.....	25
MainActivity.kt.....	25
8.4 Diseño UI.....	25
Filosofía de diseño y navegación.....	25
Jerarquía visual.....	26
Feedback inmediato.....	26
Pantallas principales.....	26
Autenticación y registro (AuthScreen).....	26
Modo claro.....	26
Modo Oscuro.....	27
Lista de Torneos (HomeScreen).....	27
Modo Claro.....	28
Modo Oscuro.....	29
Ajustes de la aplicación (SettingsScreen).....	29
Modo Claro.....	30
Modo Oscuro.....	30
Perfil de Usuario (ProfileScreen).....	30
Modo claro.....	31
Gestión de usuarios y reputación (UsersScreen).....	31
Modo Claro.....	32
Modo Oscuro.....	32
ModalNavigationDrawer.....	32
Modo Claro.....	33
Modo Oscuro.....	33
Componentes reutilizables.....	34
8.5 Diagramas.....	35
Diagrama de Clases.....	35
Diagrama de Casos de Uso.....	36
9. Instalación y preparación.....	36
9.1. Procedimientos necesarios para hacer funcionar el proyecto.....	36
9.2. Procedimientos necesarios para el control de versiones.....	37
9.3. Procedimientos para el registro y resolución de errores.....	37
10. Documentación de ejecución y plan de calidad.....	38
10.1. Procedimientos operativos.....	38
10.2. Registro de pruebas.....	38

10.3. Indicadores de calidad.....	39
10.4. Métodos de verificación.....	39
11. Distribución.....	40
11.1. Tecnología de distribución.....	40
11.2. Descripción del proceso.....	40
12. Manuales.....	41
12.1. Manual de instalación.....	41
12.2. Manual de uso (Guía del usuario).....	42
13. Conclusiones.....	43
13.1. Informe final.....	43
13.2. Resultados esperados.....	43
13.3. Viabilidad del proyecto.....	44
13.4. Mejoras futuras.....	44
14. Anexos.....	45
14.1 Registro de Pruebas.....	45
PR-01 (Autenticación).....	45
PR-02 (Validación de Autenticación).....	45
PR-03 (Inscripción de torneo).....	46
PR-04 (Concurrencia).....	47
PR-05: Seguridad RLS.....	48
PR-06 (Reputación).....	49
14.2 Índice de imágenes y tablas.....	50
Índice de imágenes.....	50
Índice de tablas.....	50
15. Bibliografía.....	51

1. Introducción

1.1 Justificación del Proyecto

Actualmente, el sector de los videojuegos y los eSports ha experimentado un crecimiento significativo, generando la necesidad de herramientas digitales que faciliten la organización y participación en competiciones. Gestionar estas ligas o torneos es una tarea monumental que va mucho más allá de emparejar equipos y programar partidas. Muchas plataformas existentes presentan interfaces complejas o están orientadas a entornos profesionales, lo que dificulta su uso por parte de jugadores amateurs o pequeños organizadores.

La finalidad del proyecto es ofrecer una solución accesible y fácil de utilizar que centralice la información de torneos y simplifique el proceso de inscripción y seguimiento para los usuarios permitiendo a los organizadores enfocarse en lo que realmente importa, la competición y la experiencia del jugador.

La elección de este proyecto responde tanto al interés personal en el ámbito de los videojuegos como a la posibilidad de aplicar de forma práctica lo aprendido gestionando bases de datos, aplicando y usando tecnologías de desarrollo móvil y servicios en la nube. Se contempla la posibilidad de financiar la aplicación mediante anuncios, lo que permitiría cubrir costes de mantenimiento de la base de datos en la nube o futuras mejoras.

1.2 Análisis Comparativo de Aplicaciones Similares

En esta fase inicial se analizaron otras plataformas que ya proveen algo similar a lo que se busca con la creación de este proyecto. Hay varias soluciones que ofrecen un amplio abanico de características con necesidades y tamaños distintos.

Actualmente estas son clave en los torneos competitivos actualmente.

- EsportsHub Pro: Esta plataforma se ha posicionado como un referente gracias a su enfoque integral y su capacidad para manejar la complejidad de los torneos de alto nivel permitiendo desde el registro de equipo hasta la administración de partidas.
- LeagueMaster: Se distingue por su enfoque en la optimización del rendimiento y la experiencia del usuario a través de funciones muy especializadas. Está diseñada para ligas que ya tienen una base de jugadores y buscan herramientas que permitan mejorar su gestión.
- TourneyFlow: Esta es la que más se acerca a lo que buscamos. Ya que busca ser perfecta para aquellos que buscan priorizar la facilidad de uso sin sacrificar eficacia. Tiene una interfaz intuitiva que hace que la gestión sea fácil y accesible para los que no tienen una experiencia técnica o conocimientos amplios y avanzados.

- NexusForge: Esta plataforma se destaca por ofrecer una amplia personalización. Importante para organizaciones con una imagen fuerte que necesitan funcionalidades específicas. Permite a los organizadores alinear su visión de la marca con lo necesario para su liga.
- ArenaConnect: El foco se encuentra en construir una comunidad siendo una muy buena herramienta para la interacción de usuarios. No solo administra torneos sino que les da un espacio a los jugadores para poder interactuar entre sí y sentirse parte de un grupo cada vez más grande y significativo.

Analizando estas plataformas se puede deducir de que la mayoría son para entornos profesionales o marcas grandes que quieran organizar sus torneos. Salvo el caso de TourneyFlow hay pocas posibilidades para torneos amateurs que no tengan una base tan grande. Además están enfocadas al entorno web y no ofrecen una experiencia para dispositivos móviles. Se plantea otorgar facilidades a este tipo de torneos además de ofrecer una plataforma móvil útil e intuitiva para la gestión de los mismos.

1.3 Tendencias Actuales

El sector de los videojuegos es un sector que no deja de evolucionar en ningún momento. Actualmente muchas organizaciones realizan torneos ya sea para promocionar sus productos como para entretener al público como por ejemplo Twitch, Youtube, Amazon o incluso empresas de videojuegos que hace sus propios torneos sobre sus propios juegos como Riot, Valve o Epic Games.

El desarrollo de aplicaciones con conexión a servicios en la nube permite crear plataformas accesibles desde cualquier dispositivo y que estas sean sincronizadas en tiempo real. Permiten que tecnologías como bases de datos cloud y autenticación online faciliten la creación de estas aplicaciones multiplataforma. Por ende estas tendencias actuales favorecen la creación de una solución móvil para simplificar la gestión de este tipo de eventos digitales y fomenten la interacción de usuarios.

1.4 Beneficios y Expectativas

Por ende queremos aportar los siguientes beneficios a aquellos que usen nuestra plataforma:

- Facilitar la organización de torneos de videojuegos enfocado al entorno amateur.
- Centralizar la información de usuarios y competidores en una única aplicación.
- Ofrecer una experiencia sencilla e intuitiva adaptada a dispositivos móviles.

- Permitir que los usuarios vinculen sus nombres de usuario de Steam y Riot en su perfil, reforzando su identidad y que otros usuarios puedan añadirle a sus listas de amigos en las plataformas respectivas.

2. Descripción del Proyecto

2.1 Tipo de Proyecto

El proyecto va a consistir en el desarrollo de una aplicación móvil Android conectada a un backend en la nube utilizando la plataforma Supabase siguiendo una arquitectura cliente-servidor. Esta aplicación almacenará la información online permitiendo a los usuarios acceder a los mismos datos desde distintos dispositivos en tiempo real.

2.2 Características Principales

La aplicación está pensada para tener una serie de funcionalidades principales:

- Registro e inicio de sesión mediante una autenticación online.
- Se diferenciaron varios roles con algunas diferencias entre sí. Jugador, Organizador y Admin.
- Visualización de torneos disponibles.
- Creación de torneos por parte de los usuarios de manera limitada.
- Inscripción en torneos y control de participantes.
- Sistema de feedback post-evento.
- Sistema de reputación del organizador.
- Sincronización en tiempo real con Supabase.
- Los usuarios podrán agregar sus nombres de usuario de Steam y Riot en su perfil.

2.3 Usuarios destinatarios

La aplicación está enfocada principalmente a la siguiente base de usuarios:

- Jugadores que deseen participar en torneos de videojuegos.
- Comunidades gaming pequeñas o grupos de usuarios que quieran hacer torneos de manera sencilla.

Inicialmente no está enfocado a competiciones profesionales sino a un entorno accesible para pequeños grupos.

3. Objetivos del Proyecto

3.1 Objetivo General

La idea del proyecto es la creación de una aplicación móvil multiplataforma que permita a los usuarios realizar la creación, organización e inscripción de torneos de videojuegos de manera sencilla e intuitiva.

3.2 Objetivos Específicos

1. Permitir el registro e inicio de sesión de usuarios de manera segura usando Supabase.
2. Crear, editar y gestionar torneos por parte de los usuarios, con un límite inicial para evitar spam.
3. Registrar la inscripción de participantes y controlar los cupos disponibles en cada torneo.
4. Desarrollar un sistema de feedback que permita calcular de forma automática la valoración media de los torneos y la reputación global de los jugadores y organizadores.
5. Sincronizar todos los datos en tiempo real mediante Supabase, accesibles desde cualquier dispositivo.
6. Diseñar una interfaz de usuario priorizando facilidad de uso y navegación intuitiva.

4. Alcance del proyecto

Incluye

- Desarrollo de una aplicación móvil Android que permita a los usuarios organizar, participar y gestionar torneos de videojuegos.
- Backend en Supabase para el almacenamiento de usuarios, torneos, inscripciones y reputación.
- Creación y gestión de torneos por parte de los usuarios, con limitaciones iniciales para evitar spam.
- Registro de inscripciones de participantes y control de cupos disponibles.
- Sistema de puntuación y valoración de torneos mediante estrellas, una vez finalizados.
- Sistema de reputación y desbloqueo progresivo de torneos según la actividad y valoración de cada usuario.
- Sincronización en tiempo real de datos entre dispositivos.
- Roles de usuario: participante, organizador y administrador, con permisos diferenciados.

- Visualización de torneos ordenados por fecha, número de inscritos o puntuación media.

Límites / restricciones

- La aplicación se centra únicamente en dispositivos móviles Android.
- La seguridad se basa en las reglas de acceso de Supabase, sin autenticaciones externas adicionales.
- La monetización mediante anuncios se contempla únicamente de manera teórica dado que la base de usuarios actual es limitada y no generará ingresos reales en esta versión del proyecto.
- La verificación de cuentas externas (Steam o Riot) será únicamente informativa, sin autenticación oficial mediante sus APIs.
- El sistema de moderación se implementará de forma básica y no incluirá herramientas avanzadas de gestión comunitaria.

5. Requisitos del Proyecto

5.1 Requisitos Funcionales

Los requisitos funcionales describen las funcionalidades que deberá ofrecer la aplicación:

- El sistema permitirá el registro de usuarios mediante correo electrónico y contraseña.
- El sistema permitirá el inicio y cierre de sesión de usuarios autenticados.
- Los usuarios podrán visualizar una lista de torneos disponibles.
- Los usuarios podrán consultar la información detallada de cada torneo.
- Los usuarios podrán inscribirse en torneos siempre que existan plazas disponibles.
- El sistema comprobará automáticamente la disponibilidad de cupos antes de permitir la inscripción.
- Los usuarios podrán crear torneos con un límite inicial para evitar spam.
- Los organizadores podrán eliminar sus torneos.
- El sistema permitirá a los usuarios valorar los torneos una vez marcados como 'finalizados', siempre que el usuario haya estado inscrito en dicho torneo.
- El sistema calculará automáticamente la puntuación media del torneo y actualizará la reputación del organizador basándose en las nuevas valoraciones recibidas.
- Los usuarios podrán consultar su historial de torneos e inscripciones
- Los administradores podrán moderar contenidos y gestionar usuarios o torneos.

- Los usuarios podrán añadir sus identificadores de plataformas externas (Steam, Riot) a su perfil.

5.2 Requisitos Técnicos

Los requisitos técnicos definen las tecnologías y condiciones necesarias para el desarrollo del sistema:

- Plataforma: Android (API 36) o superior.
- Lenguaje y Framework: Kotlin con Jetpack Compose (UI Declarativa).
- Backend: Supabase (BaaS) integrando base de datos PostgreSQL.
- Comunicación: Arquitectura REST y WebSockets mediante cliente Ktor.
- Seguridad: Autenticación JWT y políticas de acceso a nivel de fila (RLS) en servidor.
- Persistencia: Serialización de datos mediante Kotlinx Serialization.
- Asincronía: Gestión de hilos mediante Kotlin Coroutines para mantener la fluidez de la interfaz (60 FPS).

5.3 Requisitos Legales o Normativos

La aplicación EasyTournament ha sido diseñada cumpliendo con el marco legal vigente en materia de protección de datos y servicios digitales:

- RGPD y LOPDGDD: Se implementa un mecanismo de consentimiento informado en el registro. Los datos personales recogidos se limitan a los estrictamente necesarios para la operatividad del servicio. Se garantiza el derecho de supresión mediante el borrado en cascada en la base de datos.
- Seguridad de la Información: La persistencia de datos cumple con estándares de seguridad mediante el uso de tokens JWT y políticas de seguridad a nivel de fila (RLS), impidiendo el acceso no autorizado a información sensible.
- Términos de Uso: El usuario acepta las reglas de la comunidad y el sistema de reputación, el cual actúa como moderador automático para garantizar un entorno seguro y libre de fraudes en los torneos.
- Propiedad Intelectual: El software se desarrolla bajo principios de código abierto, utilizando librerías externas que respetan sus correspondientes licencias (Apache/MIT).

6. Planificación del proyecto

6.1 Estructura de tareas

Diseño del Proyecto

- Análisis de requisitos y planificación del proyecto
- Diseño de la arquitectura general

- Diseño del esquema de base de datos en PostgreSQL
- Definición de relaciones y reglas de negocio

Desarrollo de Backend

- Configuración del backend con Supabase
- Implementación de autenticación y almacenamiento
- Políticas Row Level Security (RLS)
- Disparadores de servidor (Triggers)

Desarrollo del Núcleo de la Aplicación Android (Frontend)

- Configuración del Stack Tecnológico
- Capa de Persistencia
- Gestión de Navegación

Diseño de Interfaz y Experiencia de Usuario (UX)

- Arquitectura de UI Declarativa y Sistema de Temas
- Lógica adaptativa: Modo Claro y Modo Oscuro
- Navegación Avanzada y Feedback

Pruebas de Calidad, Depuración y Despliegue

- Pruebas de Lógica de Negocio (Backend)
- QA de Interfaz (Usabilidad y Compatibilidad)
- Documentación Final y Generación de APK

6.2 Cronograma de Gantt

<https://drive.google.com/file/d/1Krb0RBSWSfYUKi4edEmF04tCs2VxOtb2/view?usp=sharing>

6.3 Recursos Necesarios

Recursos de Hardware

- Estación de trabajo: PC con arquitectura x86_64, procesador con soporte de virtualización (para el emulador de Android) y un mínimo de 16 GB de memoria RAM.
- Dispositivo de pruebas: Smartphone físico Android y emulador de Android Studio configurado con una imagen de sistema API 36 como base mínima de compatibilidad.

Recursos de Software (Stack Tecnológico)

- Este proyecto se basa en un ecosistema de desarrollo que garantiza el rendimiento y la mantenibilidad del código:
- Entorno de Desarrollo (IDE): Android Studio.

- Lenguaje de programación: Kotlin 1.9.22, haciendo uso intensivo de corrutinas para la gestión de hilos asíncronos y Kotlinx Serialization para el parseo de datos JSON.
- Interfaz de Usuario: Jetpack Compose (librería androidx.compose.material3), utilizando un modelo de UI declarativa y componentes de Material Design 3.
- Gestión de Red: Cliente Ktor (io.ktor:ktor-client-android) para la comunicación mediante protocolos HTTP/REST.
- Navegación: Uso de androidx.navigation:navigation-compose para el control de la pila de pantallas y flujo de la aplicación.

Recursos de Infraestructura y Backend (Cloud)

- BaaS (Supabase): Se utiliza como infraestructura principal para evitar el mantenimiento de un servidor propio.
- Base de datos: Motor de PostgreSQL hospedado en la nube.
- Autenticación: Servicio de autenticación Supabase para la gestión de identidades y persistencia de sesión.
- Seguridad: Implementación de políticas RLS y funciones en PL/pgSQL para la lógica de servidor.
- Control de Versiones: Git como sistema de control de versiones local y GitHub como repositorio remoto para la gestión del código fuente.

Recursos Humanos

- Desarrollador Full-stack: Una persona encargada de todo el ciclo de vida del software.

7. Plan de gestión de riesgos

En este apartado se analizan las posibles amenazas que podrían comprometer el desarrollo o la operatividad de EasyTournament, estableciendo medidas preventivas y soluciones de contingencia.

7.1 Identificación y evaluación de riesgos

Riesgo	Impacto	Probabilidad	Descripción
Pérdida de conectividad	Alto	Media	La aplicación depende completamente del backend cloud para su funcionamiento.
Inconsistencia de datos	Crítico	Baja	Posibles conflictos cuando varios usuarios intentan ocupar simultáneamente la última plaza disponible de un torneo.

Acceso no autorizado	Crítico	Baja	Riesgo de manipulación de perfiles, inscripciones o valoraciones de otros usuarios.
Incompatibilidad de hardware	Medio	Baja	Posibles errores de funcionamiento en dispositivos Android antiguos o con recursos limitados.
Spameo de torneos	Medio	Baja	Posibilidad de que los usuarios creen gran cantidad de torneos vacíos.

Tabla 1 Tabla de Riesgos

7.2 Recursos preventivos

Para minimizar las probabilidades de fallo, se han implementado las siguientes medidas preventivas durante el desarrollo:

- Integridad referencial en servidor: Uso de claves foráneas y restricciones ON DELETE CASCADE para evitar registros huérfanos en la base de datos.
- Políticas RLS (Row Level Security): Las reglas de acceso se ejecutan directamente en PostgreSQL mediante Supabase, evitando depender únicamente de validaciones en el cliente Android. Previendo cosas como el spam.
- Validación de datos en tiempo real: Implementación de validaciones en formularios de autenticación y registro, como comprobación del formato de correo electrónico y longitud mínima de contraseña y de nombre de usuario.
- Uso de identificadores UUID: Los identificadores universales reducen el riesgo de ataques por enumeración o predicción de IDs de usuarios y torneos.
- Compatibilidad mínima controlada: La aplicación se desarrolla tomando como referencia Android API 36 para garantizar compatibilidad con una mayor variedad de dispositivos.

7.3 Plan para mitigar las consecuencias (Contingencia)

En caso de que alguno de los riesgos se materialice, se aplicarán las siguientes medidas de contingencia:

- Ante concurrencia masiva en inscripciones: Los triggers y validaciones en servidor controlan el número máximo de participantes de forma atómica. Si dos usuarios intentan ocupar la última plaza simultáneamente, la base de datos rechazará automáticamente una de las operaciones y devolverá un error controlado.

- Ante fallos de red o pérdida de conectividad: Para la gestión de fallos de conectividad, se ha implementado un sistema de filtrado de mensajes de error. El sistema no solo captura excepciones genéricas, sino que analiza las cadenas de error del SDK (cómo 'Unable to resolve host') para traducirlas a un lenguaje comprensible para el usuario final.
- Ante pérdida o expiración de sesión: El sistema de autenticación invalidará automáticamente la sesión caducada y redirigirá al usuario a la pantalla de inicio de sesión de forma segura.
- Ante errores de lógica o datos inesperados: El uso de Kotlinx Serialization permite detectar errores de pareo o estructuras JSON inválidas sin provocar fallos críticos en la aplicación.
- Ante accesos no autorizados: Las políticas RLS impedirán operaciones no permitidas sobre perfiles, torneos, inscripciones o reseñas, incluso si se intenta manipular directamente la API.

8. Diseño

8.1. Prototipado (Wireframe de la aplicación)

La interfaz de EasyTournament ha sido diseñada siguiendo los principios de Material Design 3, adoptando una estética orientada al entorno gaming y ofreciendo soporte nativo para modo oscuro y modo claro mediante una paleta de colores personalizada definida en Theme.kt.

La navegación principal se organiza mediante un Navigation Drawer (menú lateral), permitiendo adaptar la experiencia según el rol del usuario dentro de la plataforma.

Autenticación y registro (AuthScreen)

Pantalla encargada de la autenticación de usuarios mediante Supabase Auth.

Funciones principales:

- Inicio de sesión
- Registro de nuevos usuarios
- Validación de formularios
- Aceptación de términos RGPD (fun TermsAndConditionsDialog)

Durante el registro, un trigger de PostgreSQL (handle_new_user) genera automáticamente el perfil correspondiente en la tabla profiles.

Además, se aplican las siguientes validaciones:

- Username mínimo de 5 caracteres
- Contraseña mínima de 6 caracteres
- Verificación del formato de email

Exploración de torneos (HomeScreen)

Pantalla principal donde se muestran los torneos almacenados en la tabla tournaments.

Cada tarjeta de torneo incluye:

- Nombre del torneo
- Juego
- Número de participantes
- Fecha
- Modalidad
- Organizador
- Estado actual

El estado del torneo se actualiza automáticamente entre:

- Abierto
- Cerrado
- Finalizado

La inscripción de usuarios se realiza en tiempo real, actualizando automáticamente el contador current_participants.

Funciones principales:

- Mostrar lista de torneos en formato de cards. (TournamentList y TournamentCard)
- Mostrar detalles de cada torneo (TournamentDetailDialog)
- Crear torneos (CreateTournamentDialog)

Gestión de Usuarios (UsersScreen)

Vista exclusiva para administradores de la plataforma.

Funciones principales:

- Buscar perfiles registrados
- Ver Usuario de Steam y Riot (InfoRow)
- Gestionar reputación de usuarios

Los administradores se identifican visualmente mediante un icono de estrella dorada. Al tocar en un usuario de la lista accederán a su ProfileScreen.

Detalle de Perfil (ProfileScreen)

Pantalla destinada a visualizar la información del usuario. En caso de ser el usuario autenticado en el momento podrá cambiar sus datos. Estos son:

- Nombre de usuario
- Reputación actual
- Cuenta vinculada de Steam
- Cuenta vinculada de Riot Games

La reputación puede ser positiva o negativa dependiendo de las acciones administrativas aplicadas sobre el usuario.

Configuración y Personalización (SettingsScreen)

La pantalla de ajustes permite al usuario adaptar la experiencia de la aplicación según sus preferencias personales. Su diseño sigue el esquema tradicional de “Lista de Opciones” utilizado en Android, ofreciendo una navegación intuitiva y familiar para el usuario.

Gestión del tema

La funcionalidad principal de esta pantalla es la gestión dinámica del tema visual de la aplicación mediante un componente Switch.

El usuario puede alternar entre:

- Modo Claro
- Modo Oscuro

La preferencia seleccionada se almacena localmente utilizando SharedPreferences, permitiendo conservar la configuración incluso después de cerrar completamente la aplicación.

La lógica de persistencia y aplicación del tema se gestiona desde MainActivity.kt, garantizando que el tema seleccionado se aplique automáticamente durante el inicio de la aplicación.

Idioma (no implementado)

La aplicación actualmente solo está en idioma español. Se contempla a futuro poder cambiar el idioma a otros como inglés y francés y extenderse a futuro a otros.

Notificaciones (no implementado)

Aún no se han implementado las notificaciones pero se prevé a futuro implementar algunas cuando:

- Un torneo se ha llenado.
- Un torneo se ha cancelado.
- Un torneo ha finalizado.
- Un torneo empieza ese día.
- Un torneo empieza al día siguiente.

8.2 Modelo de Datos

1. Entidad Profile (Usuario)

Representa la información extendida del usuario dentro del sistema, vinculada directamente con la autenticación.

Campo:

- id(PK): identificador único del usuario (UUID, relacionado con Auth)
- username: nombre visible del usuario en la plataforma
- reputation: puntuación del usuario basada en su actividad
- is_admin: indicador de permisos de administración
- steam_username: usuario de steam
- riot_username: usuario de riot
- avatar_url: no implementado (futuro uso para fotos de perfil)
- created_at: fecha de creación

Los campos (steam_username y riot_username) permiten la integración informativa con plataformas externas, sin autenticación directa, únicamente con fines de perfilado del usuario.

Propósito:

Se utiliza para gestionar roles y permisos dentro de la aplicación, así como para identificar a los organizadores de torneos y mostrar su reputación dentro de la comunidad.

2. Entidad Tournament (Torneo)

Es la entidad principal del sistema y la más compleja, ya que concentra gran parte de la lógica de negocio de la aplicación.

Campos clave:

- id(PK): identificador de torneo
- creator_id: id de organizador
- title: nombre del torneo
- description: descripción del torneo
- game_name: videojuego asociado
- max_participants: número máximo de participantes
- current_participants: contador actual de inscritos
- status: estado del torneo (abierto, cerrado, finalizado)
- start_date: fecha de inicio
- rating_avg: media de valoraciones (tipo Float)
- category: Categoría del torneo (Juegos de Mesa, Videojuegos, Deportes u Otros)
- Modalidad: Forma de competición (Online o Presencial)
- created_at: fecha de creación

Relación:

Incluye una relación opcional con la entidad Profile (creador del torneo), permitiendo obtener en una sola consulta el usuario organizador mediante un JOIN. Esto optimiza el rendimiento y reduce llamadas a la base de datos.

Propósito:

Gestionar la creación, disponibilidad, estado y evaluación de los torneos dentro de la plataforma.

3. Entidad Enrollment (Inscripción)

Representa la tabla intermedia que establece la relación entre usuarios y torneos (relación N:M).

Campos clave:

- id(PK): identificador de inscripción
- tournament_id: identificador del torneo
- user_id: identificador del usuario inscrito
- enrolled_at: fecha de inscripción

Propósito:

Permite gestionar las inscripciones de los usuarios en los torneos, así como verificar si un usuario ya está registrado en una competición y controlar altas y bajas de forma segura.

4. Entidad Review (Reseña)

Almacena las valoraciones y comentarios realizados por los usuarios sobre los torneos finalizados.

Campos clave:

- id(PK): identificador review
- tournament_id: identificador de torneo al que se hace review
- reviewer_id: identificador de usuario que hace review
- rating: puntuación numérica (entero)
- comment: comentario textual del usuario
- created_at: fecha de creación

Propósito:

Alimenta los mecanismos automáticos del sistema, como el trigger de actualización de media de puntuación del torneo y la actualización de la reputación del organizador, garantizando una evaluación dinámica basada en la actividad real de la comunidad.

8.3. Especificaciones Técnicas

EasyTournament utiliza un stack tecnológico moderno orientado al desarrollo móvil nativo y a la sincronización en tiempo real entre cliente y servidor.

Tecnologías y arquitectura

1. Patrón de diseño: La aplicación utiliza una arquitectura MVVM (Model-View-ViewModel). Además, se implementa el Pattern Repository mediante:
 - a. UserRepository
 - b. TournamentRepository
 - c. AuthRepository

Esto permite separar la lógica de acceso a Supabase de la interfaz desarrollada con Jetpack Compose.

2. Asincronía: Se emplean Kotlin Coroutines y rememberCoroutineScope para realizar operaciones de red sin bloquear el hilo principal de la interfaz. Esto garantiza:
 - a. Fluidez visual
 - b. Mejor experiencia de usuario
 - c. Operaciones asíncronas eficientes
3. Serialización: Se utiliza kotlinox.serialization para transformar los datos JSON provenientes de PostgreSQL en objetos Kotlin de forma segura y tipada.
4. Backend: Se basa en Supabase utilizando PostgreSQL como motor principal de base de datos.

Lógica de negocio en servidor

EasyTournament delega parte de la lógica crítica directamente en el servidor para aumentar la seguridad y mantener la integridad de los datos.

1. Funciones: Uso de funciones como update_user_reputation para modificar reputaciones mediante llamadas seguras desde la aplicación.

Crear perfil automáticamente al registrarse en Auth.

```
CREATE OR REPLACE FUNCTION public.handle_new_user()  
RETURNS trigger AS $$  
BEGIN  
    INSERT INTO public.profiles (id, username, is_admin, reputation)  
    VALUES (
```

```

    new.id,
    COALESCE(new.raw_user_meta_data->>'username', split_part(new.email, '@',
1)),
    false,
    0.0
)
ON CONFLICT (id) DO NOTHING;
RETURN new;
END;
$$ LANGUAGE plpgsql SECURITY DEFINER;

```

Contador de participantes automático.

```

CREATE OR REPLACE FUNCTION public.update_tournament_participant_count()
RETURNS TRIGGER AS $$
BEGIN
    IF (TG_OP = 'INSERT') THEN
        UPDATE public.tournaments
        SET current_participants = current_participants + 1,
            status = CASE WHEN (current_participants + 1) >= max_participants THEN
'cerrado' ELSE 'abierto' END
        WHERE id = NEW.tournament_id;
    ELSIF (TG_OP = 'DELETE') THEN
        UPDATE public.tournaments
        SET current_participants = GREATEST(0, current_participants - 1),
            status = 'abierto'
        WHERE id = OLD.tournament_id;
    END IF;
    RETURN NULL;
END;
$$ LANGUAGE plpgsql SECURITY DEFINER;

```

Gestión de Reputación (Usada por Administradores)

```

CREATE OR REPLACE FUNCTION public.update_user_reputation(user_uuid
UUID, amount FLOAT)
RETURNS void AS $$
BEGIN
    UPDATE public.profiles
    SET reputation = reputation + amount
    WHERE id = user_uuid;
END;
$$ LANGUAGE plpgsql SECURITY DEFINER;

```

Función para actualizar la valoración del torneo

```
CREATE OR REPLACE FUNCTION public.update_tournament_rating()
RETURNS TRIGGER AS $$
BEGIN
    UPDATE public.tournaments
    SET rating_avg = (
        SELECT AVG(rating)::FLOAT
        FROM public.reviews
        WHERE tournament_id = NEW.tournament_id
    )
    WHERE id = NEW.tournament_id;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql SECURITY DEFINER;
```

2. Automatismos (Triggers): Se utilizan triggers de PostgreSQL para automatizar procesos como:
 - a. Actualización de participantes
 - b. Cambio automático de estado del torneo
 - c. Creación automática de perfiles
 - d. Gestión de reseñas y reputación

Garantiza creación de perfil al crear usuario.

```
CREATE TRIGGER on_auth_user_created
AFTER INSERT ON auth.users
FOR EACH ROW EXECUTE PROCEDURE public.handle_new_user();
```

Trigger para crear y mejorar reviews

```
CREATE TRIGGER on_review_added
AFTER INSERT OR UPDATE ON public.reviews
FOR EACH ROW EXECUTE PROCEDURE public.update_tournament_rating();
```

Garantiza aumento de contador de participantes al inscribirse

```
CREATE TRIGGER tr_update_participant_count
AFTER INSERT OR DELETE ON public.enrollments
FOR EACH ROW EXECUTE PROCEDURE
public.update_tournament_participant_count();
```

3. Seguridad RLS (Row Level Security): Las políticas RLS controlan el acceso a la base de datos directamente desde PostgreSQL. Entre las restricciones implementadas:
- a. Un usuario solo puede modificar su propio perfil
 - b. Los administradores disponen de permisos ampliados
 - c. Usuarios con reputación negativa no pueden crear torneos
 - d. Protección frente a modificaciones no autorizadas sobre torneos o reseñas.

Habilitación de Políticas

```
ALTER TABLE public.profiles ENABLE ROW LEVEL SECURITY;  
ALTER TABLE public.tournaments ENABLE ROW LEVEL SECURITY;  
ALTER TABLE public.enrollments ENABLE ROW LEVEL SECURITY;  
ALTER TABLE public.reviews ENABLE ROW LEVEL SECURITY;
```

Políticas de Perfiles

Permite lectura para todos los usuarios autenticados.

```
CREATE POLICY "Enable read access for authenticated users"  
ON public.profiles  
FOR SELECT  
TO authenticated  
USING (true);
```

Permite que el usuario actualice su propio perfil.

```
CREATE POLICY "Users can update their own profiles"  
ON public.profiles  
FOR UPDATE  
TO authenticated  
USING (auth.uid() = id);
```

Políticas de torneos

Permite la lectura de torneos para todos los usuarios.

```
CREATE POLICY "Public tournaments are viewable by everyone"  
ON public.tournaments  
FOR SELECT  
USING (true);
```

Permite insertar torneos a administradores o usuarios con reputación válida.

```
CREATE POLICY "Tournament Insertion for Admin or Reputable Users"
ON public.tournaments
FOR INSERT
TO authenticated
WITH CHECK (
  (SELECT is_admin FROM public.profiles WHERE id = auth.uid()) = true
  OR
  (SELECT reputation FROM public.profiles WHERE id = auth.uid()) >= 0
);
```

Permite eliminar torneos a administradores o al creador del torneo.

```
CREATE POLICY "Creators and Admins can delete tournaments"
ON public.tournaments
FOR DELETE
TO authenticated
USING (
  auth.uid() = creator_id
  OR
  (SELECT is_admin FROM public.profiles WHERE id = auth.uid()) = true
);
```

Políticas de Inscripciones

Lectura de inscripciones para usuarios autenticados.

```
CREATE POLICY "Users can see all inscriptions"
ON public.enrollments
FOR SELECT
TO authenticated
USING (true);
```

Permite que un usuario se inscriba a sí mismo.

```
CREATE POLICY "Users can enroll themselves"
ON public.enrollments
FOR INSERT
TO authenticated
WITH CHECK (auth.uid() = user_id);
```

Permite que un usuario se desinscriba a sí mismo.

```
CREATE POLICY "Users can unenroll themselves"  
ON public.enrollments  
FOR DELETE  
TO authenticated  
USING (auth.uid() = user_id);
```

Permite ver sólo las inscripciones propias.

```
CREATE POLICY "Users can view own enrollments"  
ON public.enrollments  
FOR SELECT  
TO authenticated  
USING (auth.uid() = user_id);
```

Políticas de Reseñas

Permite insertar reseñas para todos.

```
CREATE POLICY "Users can Read Reviews" ON public.reviews  
FOR SELECT USING (true);
```

Permite crear reseñas a sí mismo.

```
CREATE POLICY "Users can Insert their own Reviews" ON public.reviews  
FOR INSERT TO authenticated WITH CHECK (auth.uid() = reviewer_id);
```

Organización del proyecto y estructura de paquetes

La estructura del proyecto EasyTournament ha sido diseñada siguiendo un enfoque modular y escalable, con el objetivo de facilitar el mantenimiento, la reutilización de código y la separación de responsabilidades. Esta organización se inspira en principios de arquitectura limpia y en el patrón MVVM (Model–View–ViewModel) adaptado al desarrollo en Android con Kotlin y Jetpack Compose. El proyecto se divide en varios paquetes principales dentro del módulo `easytournament/`, cada uno con una responsabilidad bien definida:

`data/`

Este paquete agrupa toda la lógica relacionada con el acceso y gestión de datos de la aplicación. Se subdivide en:

- `model/`: Contiene las Data Classes con anotaciones `@Serializable`. Definen la estructura de los objetos que viajan entre la base de datos y la App.

- `network/`: incluye la configuración y cliente de conexión con servicios externos, como `SupabaseClient`, encargado de la comunicación con el backend.
- `repository/`: Centraliza la lógica de las consultas. Por ejemplo, `TournamentRepository` no solo pide datos a Supabase, sino que gestiona las reglas de inscripción y creación de eventos.

Esta capa permite desacoplar la lógica de negocio del origen de los datos, facilitando cambios futuros en la fuente de información sin afectar al resto del sistema.

`ui/`

Este paquete contiene todos los elementos relacionados con la interfaz de usuario, desarrollados principalmente con Jetpack Compose. Se organiza en:

- `screens/`: incluye las vistas principales de la aplicación (pantallas completas), donde se gestiona la interacción del usuario.
- `theme/`: define la configuración visual global de la aplicación siguiendo Material Design 3, incluyendo colores, tipografía y estilos.

Esta separación permite reutilizar componentes y mantener una interfaz consistente en toda la aplicación.

`utils/`

Aquí se incluye el [DateUtils.kt](#) para el formateo de fechas.

`MainActivity.kt`

Es el punto de entrada de la aplicación Android. Desde aquí se inicializa la interfaz principal y se gestiona la navegación entre pantallas mediante el `NavHost`. Actúa como contenedor principal del sistema de navegación basado en Compose.

8.4 Diseño UI

El diseño de interfaz de `EasyTournament` se ha desarrollado siguiendo la filosofía de Material Design 3, utilizando exclusivamente Jetpack Compose como tecnología de construcción visual.

El objetivo principal ha sido crear una aplicación funcional, moderna y visualmente orientada al entorno gaming, garantizando además accesibilidad y comodidad visual tanto en modo claro como en modo oscuro.

Filosofía de diseño y navegación

La aplicación utiliza un sistema de navegación basado en un `Modal Navigation Drawer` (menú lateral), permitiendo organizar las distintas secciones de forma clara y accesible sin sobrecargar la pantalla principal.

Las principales secciones disponibles son:

- Perfil

- Torneos
- Gestión de usuarios (solo Administración)
- Ajustes

Jerarquía visual

La información se organiza utilizando componentes Card y ListItem, permitiendo identificar rápidamente:

- Estado del torneo
- Número de participantes
- Reputación del usuario
- Información relevante de cada sección

Feedback inmediato

La aplicación incorpora indicadores visuales y mensajes informativos para mantener al usuario informado durante las operaciones de red.

Entre ellos:

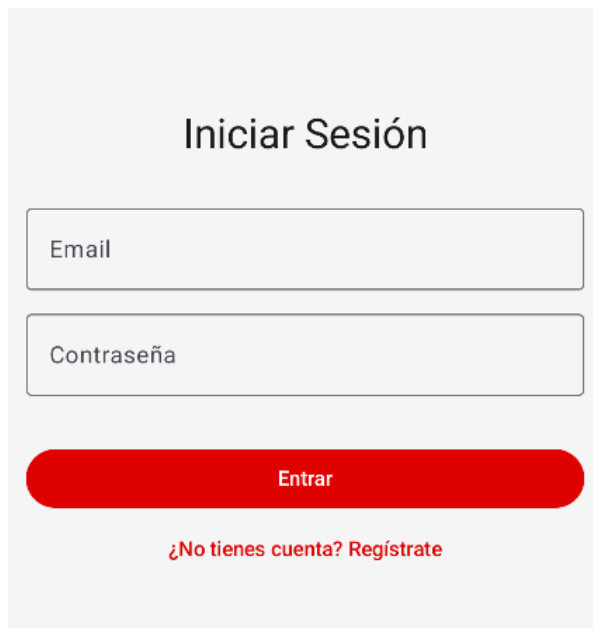
- CircularProgressIndicator
- Mensajes Toast
- Estados visuales de carga y error

Pantallas principales

Autenticación y registro (AuthScreen)

La pantalla de acceso presenta un diseño minimalista centrado en la facilidad de uso. Además, se implementa un diálogo interactivo para la aceptación de términos legales antes de permitir la creación de la cuenta.

Modo claro



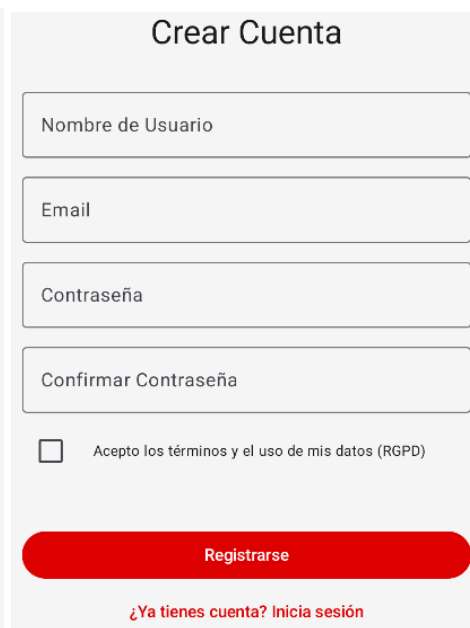
Iniciar Sesión

Email

Contraseña

Entrar

¿No tienes cuenta? [Regístrate](#)



Crear Cuenta

Nombre de Usuario

Email

Contraseña

Confirmar Contraseña

☐ Acepto los términos y el uso de mis datos (RGPD)

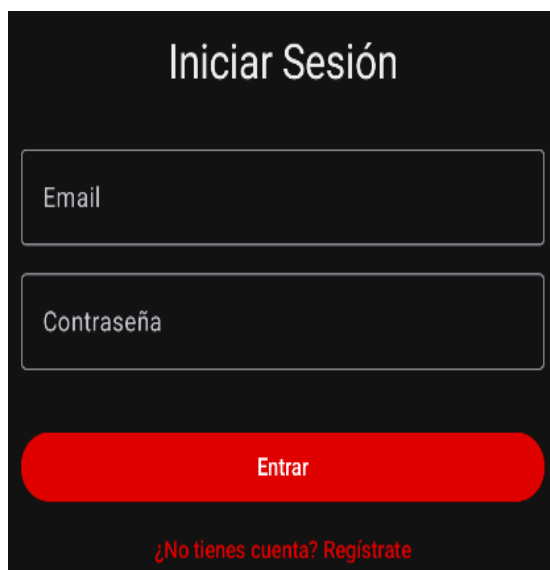
Registrarse

¿Ya tienes cuenta? [Inicia sesión](#)

Imagen 1. Iniciar Sesión Modo Claro

Imagen 2. Registrar Modo Claro

Modo Oscuro



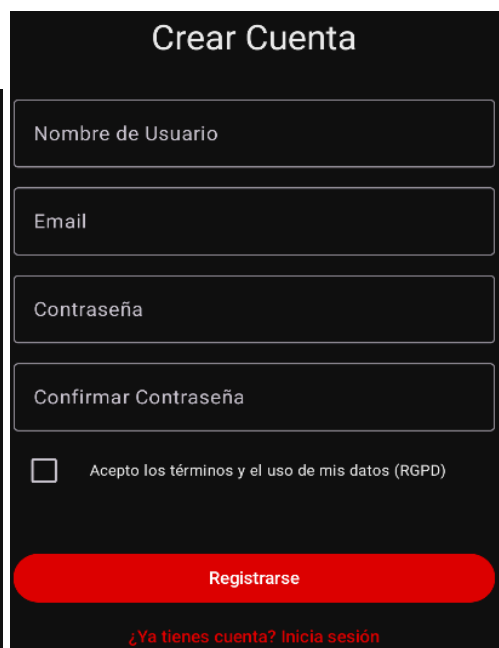
Iniciar Sesión

Email

Contraseña

Entrar

¿No tienes cuenta? [Regístrate](#)



Crear Cuenta

Nombre de Usuario

Email

Contraseña

Confirmar Contraseña

☐ Acepto los términos y el uso de mis datos (RGPD)

Registrarse

¿Ya tienes cuenta? [Inicia sesión](#)

Imagen 3. Iniciar Sesión Modo Oscuro

Imagen 4. Registrar Modo Oscuro

Lista de Torneos (HomeScreen)

La pantalla principal utiliza LazyColumn para mostrar dinámicamente los torneos activos almacenados en Supabase.

Cada tarjeta de torneo incluye:

- Juego
- Categoría
- Modalidad
- Fecha

- Estado
- Número de participantes

Modo Claro

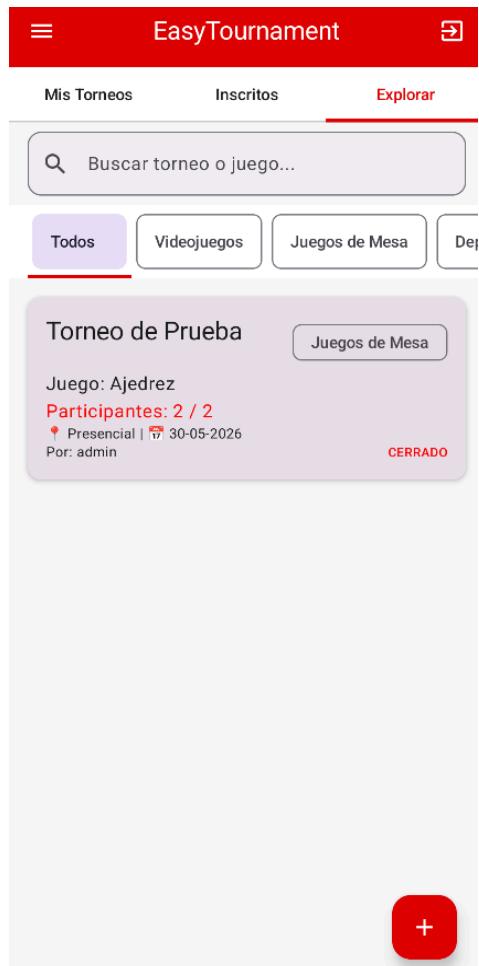


Imagen 5. HomeScreen Modo Claro

Crear Nuevo Torneo

Máx Participantes

Fecha de Inicio


Categoría


Modalidad (Online/Presencial)

Cancelar
Crear

Imagen 6. Registrar Torneo Modo Claro

Modo Oscuro

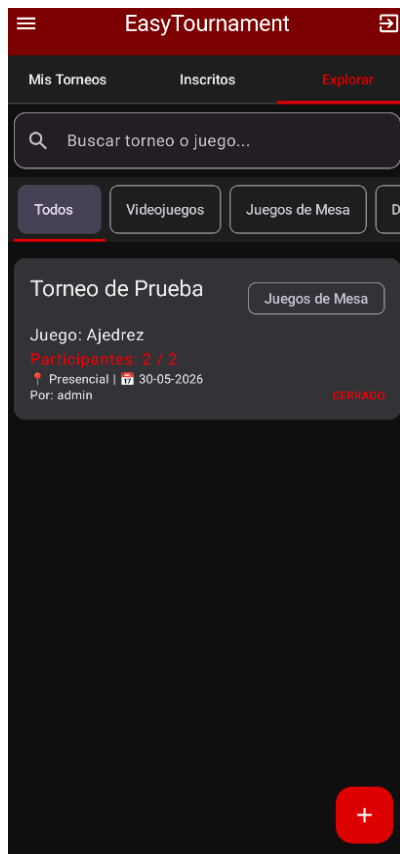


Imagen 7. HomeScreen Modo Oscuro

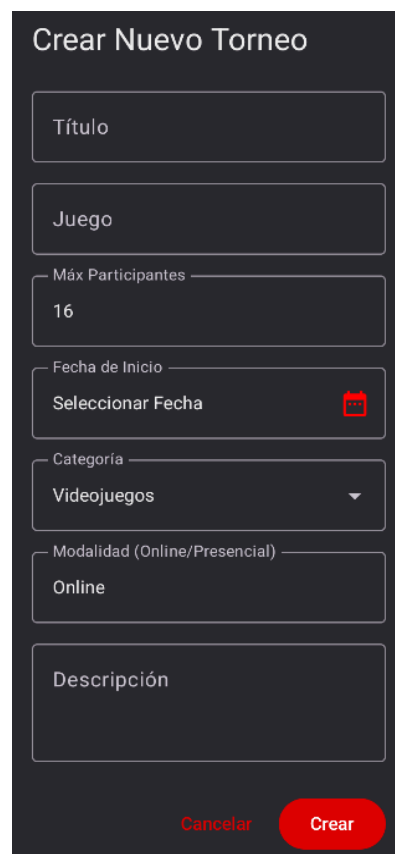


Imagen 8. Registrar Torneo Modo Oscuro

El Botón de Acción Flotante (FAB) únicamente aparece cuando el usuario dispone de reputación válida (≥ 0), restringiendo la creación de torneos a usuarios autorizados.

Ajustes de la aplicación (SettingsScreen)

Pantalla de ajustes de la aplicación. Usa componentes `ListItem` para mostrar una interfaz cómoda con las distintas opciones de configuración.

Modo Claro

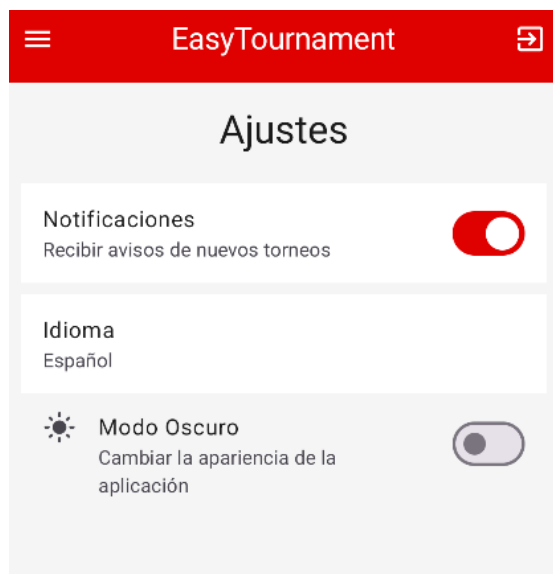


Imagen 9. SettingsScreen Modo Claro

Modo Oscuro

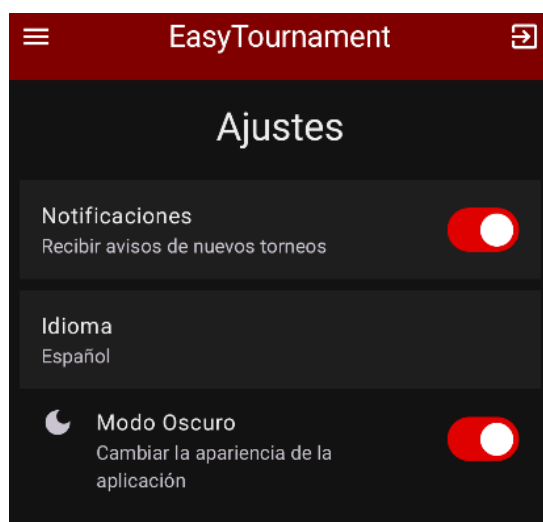


Imagen 10. SettingsScreen Modo Oscuro

Perfil de Usuario (ProfileScreen)

La pantalla de perfil permite reforzar la identidad del usuario dentro de la plataforma. En ella se visualiza:

- Nombre de usuario
- Reputación
- Cuenta de Steam
- Cuenta de Riot Games

La reputación utiliza colores dinámicos:

- Verde para reputaciones positivas
- Rojo para reputaciones negativas

Esto facilita la lectura rápida del estado del usuario dentro de la comunidad.

Modo claro

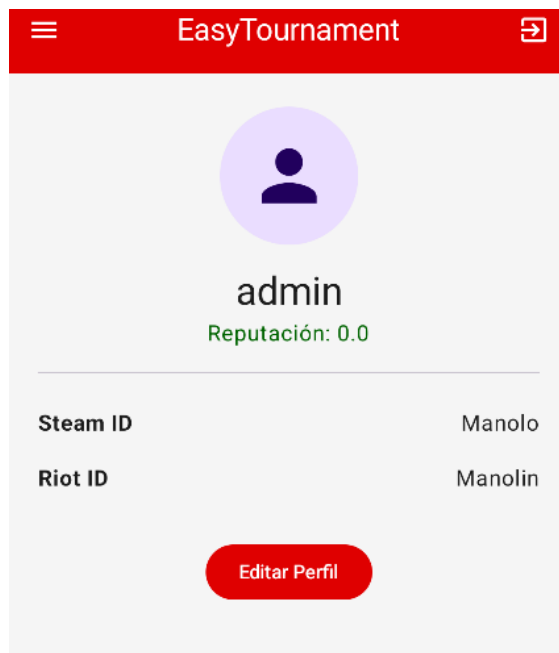


Imagen 11 ProfileScreen Modo Claro
Modo oscuro

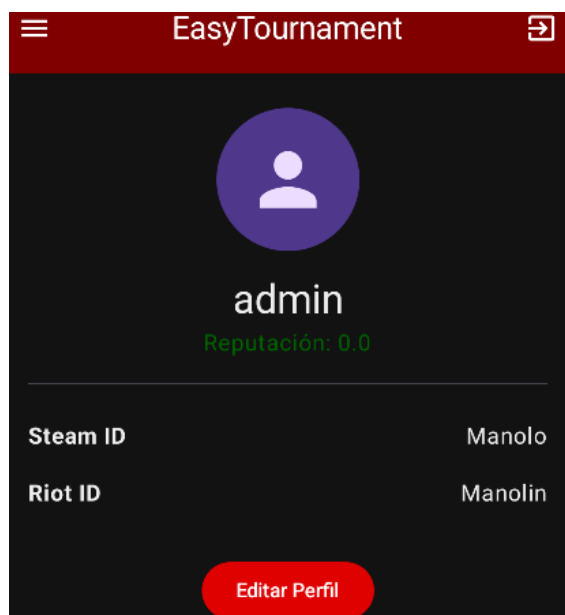


Imagen 12 ProfileScreen Modo Oscuro

Gestión de usuarios y reputación (UsersScreen)

Pantalla exclusiva para administradores de la plataforma.

Permite:

- Buscar usuarios a través de una barra de búsqueda
- Visualizar reputación.

- Modificar puntuaciones con colores dinámicos -0,5 en rojo y +0,5 en verde.

Los administradores se identifican visualmente mediante el icono Icons.Default.Star en color dorado.

Modo Claro

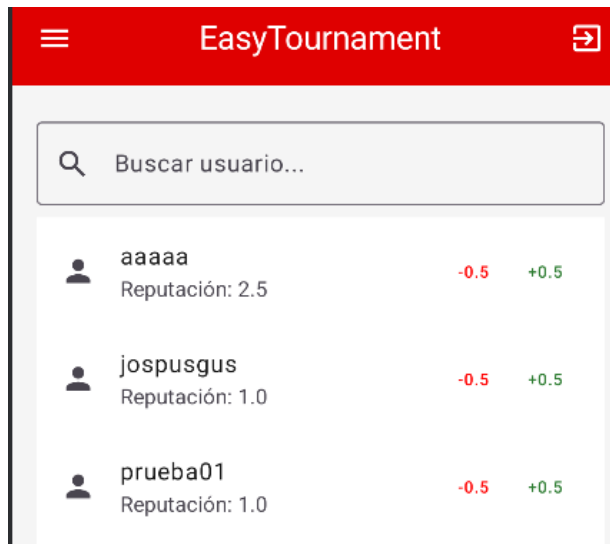


Imagen 13 UsersScreen Modo Claro

Modo Oscuro

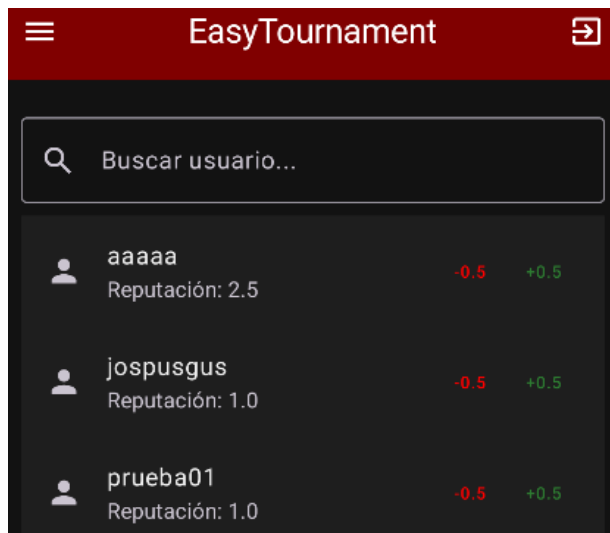


Imagen 14 UsersScreen Modo Oscuro

ModalNavigationDrawer

Al darle al botón hamburguesa se abrirá una barra lateral con los distintos campos accesibles de la aplicación.

Modo Claro

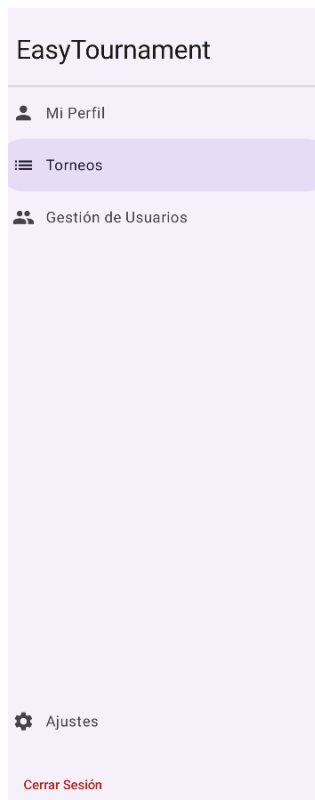


Imagen 15 ModalNavigationDrawer Modo Claro

Modo Oscuro

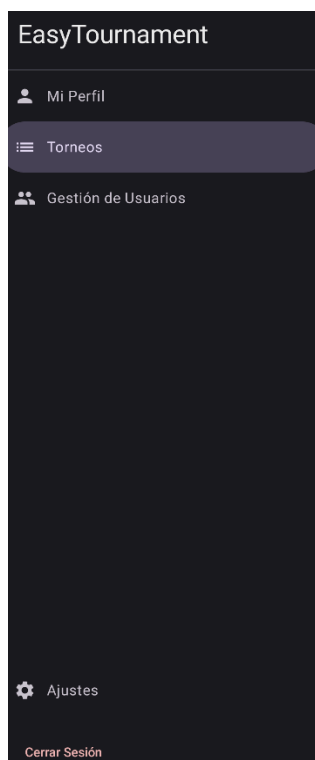


Imagen 16 ModalNavigationDrawer Modo Oscuro

Componentes reutilizables

Para mantener consistencia visual y reutilización de código, se han desarrollado componentes personalizados reutilizables en toda la aplicación:

1. TournamentCard: Es el componente visual principal de la lista de torneos. Abstrae la lógica de representación de un objeto de tipo Tournament para ser mostrado en una LazyColumn.
 - a. Es el componente visual principal de la lista de torneos. Abstrae la lógica de representación de un objeto de tipo Tournament para ser mostrado en una LazyColumn.
 - b. Lógica de Negocio Integrada:
 - i. Estados Dinámicos: Calcula visualmente si el torneo está "finalizado", "lleno" o "abierto" basándose en la fecha del sistema y el cupo de inscritos.
 - ii. Control de Acceso: Gestiona la visibilidad del botón de inscripción. Si el usuario ya es el creador (isOwner) o ya está inscrito (isEnrolled), el componente adapta sus acciones y etiquetas automáticamente.
 - iii. Formateo: Utiliza la clase DateUtils para transformar las marcas de tiempo de la base de datos en fechas legibles para el usuario.
2. El componente InfoRow: Se utiliza para estandarizar la visualización de datos de tipo "Clave-Valor" en perfiles y detalles técnicos.
 - a. Propósito: Asegurar un alineamiento perfecto y una jerarquía visual clara.
 - b. Implementación: Utiliza un contenedor Row con Arrangement.SpaceBetween, aplicando negritas a la etiqueta informativa para diferenciarla del valor dinámico.
3. ListItem (Configuración y Listas Estándar): Este componente, perteneciente a la librería de Material3, se ha utilizado para estandarizar las entradas de datos y opciones en la pantalla de ajustes (SettingsScreen) y paneles de navegación.
 - a. Propósito: Proporcionar una estructura visual consistente para opciones interactivas que requieren un icono, un texto principal y un control de acción (como un interruptor o switch).
 - b. Implementación Técnica:
 - i. headlineContent: Alberga el título de la opción, utilizando la tipografía de cuerpo de Material Design para asegurar la legibilidad.
 - ii. supportingContent: (Opcional) Se utiliza para añadir descripciones breves debajo del título, mejorando la comprensión del usuario sobre cada ajuste.
 - iii. leadingContent: Espacio dedicado a iconos descriptivos que facilitan el reconocimiento visual rápido de la opción.

- iv. trailingContent: Se integra habitualmente con componentes de control como el Switch para la gestión del Modo Oscuro, permitiendo que la interacción sea intuitiva y directa.
- c. Justificación: Al utilizar un componente estándar de la plataforma en lugar de crear uno desde cero, se garantiza que el espaciado, los colores de contraste y las áreas de pulsación cumplan con las guías de accesibilidad de Google, mejorando la experiencia de usuario (UX).

8.5 Diagramas

Diagrama de Clases

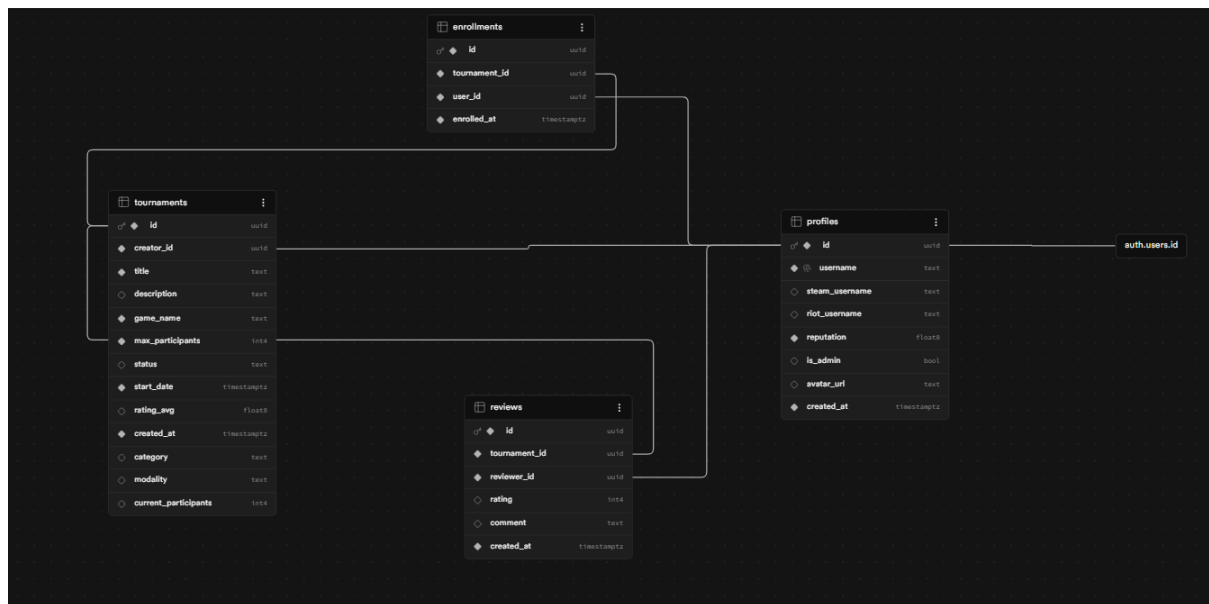


Imagen 17 Diagrama de Clases

Diagrama de Casos de Uso

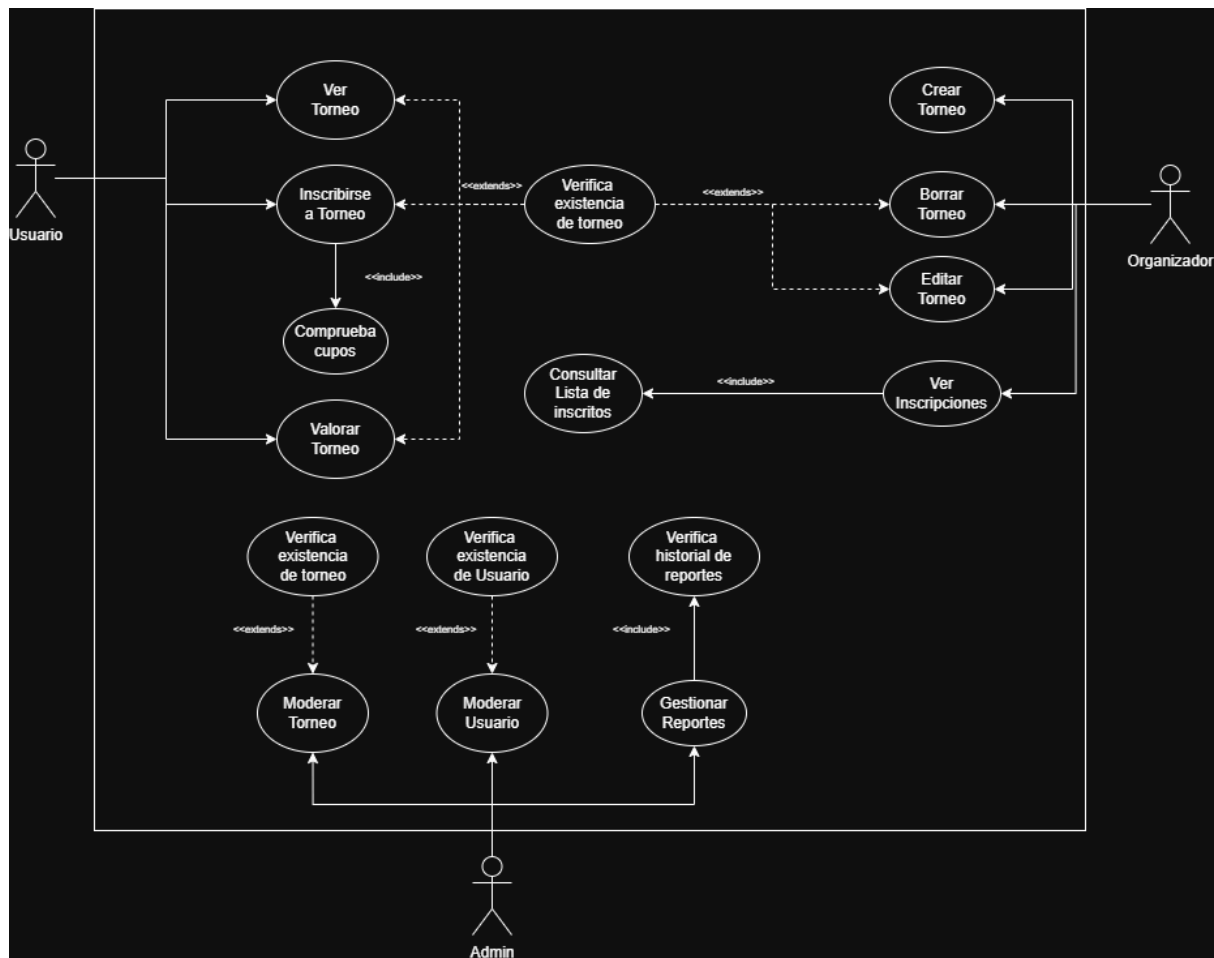


Imagen 18 Diagrama de Casos de Uso

9. Instalación y preparación

Este apartado describe el proceso para configurar el entorno de desarrollo y los protocolos seguidos para garantizar la integridad y el mantenimiento del proyecto.

9.1. Procedimientos necesarios para hacer funcionar el proyecto

Para la puesta en marcha de la aplicación desde el código fuente, el tribunal o los desarrolladores deben seguir estos pasos:

1. Entorno de Desarrollo:
 - a. Instalar Android Studio.
 - b. Configurar el SDK de Android (API 36 o superior).
2. Configuración del Backend (Supabase):
 - a. Tener acceso a un proyecto en Supabase con los servicios de Database y Auth habilitados.
 - b. La base de datos debe contener las tablas necesarias:

- i. profiles
 - ii. tournaments
 - iii. enrollments
 - iv. reviews
 - c. También deben existir los triggers necesarios para la gestión automática de perfiles y contadores de participantes.
- 3. Configuración de Credenciales:
 - a. Para facilitar la evaluación técnica y asegurar la ejecución inmediata, las credenciales de acceso (URL y API KEY de Supabase) se encuentran integradas directamente en el código fuente, específicamente dentro del paquete data.network.
 - b. Esto permite que el proyecto sea “Plug & Play”, eliminando la necesidad de configurar manualmente archivos de propiedades o variables de entorno externo antes de la primera ejecución.
- 4. Compilación y Ejecución:
 - a. Realizar una sincronización de archivos de Gradle (“Sync Project with Gradle Files”).
 - b. Desplegar la aplicación subida en GitHub en un emulador o dispositivo físico con API 36 o superior para garantizar la correcta visualización de la interfaz basada en Material 3.

9.2. Procedimientos necesarios para el control de versiones

Para la gestión del ciclo de vida del software, se ha seleccionado Git como herramienta de control de versiones distribuido:

- 1. Repositorio Remoto:
 - a. El código fuente está centralizado en un repositorio de GitHub. Esto permite disponer de un respaldo en la nube y facilidad de descarga.
- 2. Hitos de Desarrollo:
 - a. El control de versiones se organiza en registros (commits).

9.3. Procedimientos para el registro y resolución de errores

En lugar de un sistema externo de gestión, se han utilizado herramientas integradas en el entorno de desarrollo para la detección y corrección de errores en tiempo real:

- 1. Depuración mediante Logcat:
 - a. Se ha utilizado de forma intensiva el terminal Logcat de Android Studio para monitorizar el flujo de datos.
 - b. Se han implementado etiquetas de depuración para verificar la comunicación con la API y capturar excepciones de red o autenticación de manera inmediata.
- 2. Control de excepciones en código:
 - a. Se han implementado bloques try-catch en los repositorios de datos para capturar errores de conexión y mostrar mensajes informativos al

usuario, evitando que la aplicación se detenga inesperadamente (crashes).

3. Pruebas de flujo de usuario:
 - a. La detección de incidencias visuales o de navegación se ha realizado mediante pruebas iterativas directas sobre el emulador

10. Documentación de ejecución y plan de calidad

Este capítulo describe los mecanismos de control aplicados para asegurar que EasyTournament cumple con los requisitos técnicos y funcionales establecidos, garantizando una aplicación robusta y fiable.

10.1. Procedimientos operativos

Se definen como las pautas de uso y mantenimiento que aseguran el correcto funcionamiento del sistema en el día a día:

1. Monitorización del Backend:
 - a. Revisión periódica de la consola de Supabase para verificar el estado de la base de datos y la salud de los servicios de autenticación.
2. Gestión de Usuarios:
 - a. Procedimiento de revisión de perfiles para asegurar que los usuarios mantienen un comportamiento adecuado, con capacidad de gestión desde la pantalla de administrador para modificar roles o reputación.
3. Actualización de Torneos:
 - a. Los torneos pasan automáticamente de estado “abierto” a “cerrado” mediante triggers de base de datos cuando se alcanza el límite de participantes, eliminando la necesidad de intervención manual.
 - b. Los torneos pasan automáticamente a estado finalizado cuando se pasa de la fecha del torneo.

10.2. Registro de pruebas

Se han realizado pruebas de caja negra y de estrés sobre las funciones críticas de la aplicación:

ID	Funcionalidad	Descripción de la prueba	Resultado	Enlaces
PR-01	Autenticación	Registro de nuevo usuario con email válido y contraseña de 6 caracteres.	OK	Enlace

PR-02	Validación de Autenticación	Intento de registro con un email ya existente en la base de datos.	OK (Error controlado)	Enlace
PR-03	Inscripción	Un usuario se inscribe en un torneo y el contador aumenta automáticamente.	OK	Enlace
PR-04	Concurrencia	Intento de inscripción en un torneo que ya ha alcanzado su capacidad máxima.	OK (Botón deshabilitado)	Enlace
PR-05	Seguridad RLS	Intento de un usuario de borrar un torneo creado por otra persona.	OK (Botón deshabilitado)	Enlace
PR-06	Reputación	Intento de un usuario de crear un torneo con reputación negativa.	OK (Botón deshabilitado)	Enlace

Tabla 2 Pruebas de aplicación

10.3. Indicadores de calidad

Para medir el éxito del proyecto, se han establecido los siguientes indicadores (KPIs):

1. Tiempo de respuesta: Las peticiones a la API de Supabase, como la carga de torneos, no deben exceder los 2 segundos bajo condiciones normales de red.
2. Tasa de errores (Crashes): El objetivo es mantener un 0% de cierres inesperados durante el flujo principal:
 - Login
 - Home
 - Inscripción en torneos
3. Usabilidad: El usuario debe poder inscribirse en un torneo en menos de 3 clics desde la autenticación.
4. Integridad de datos: El 100% de los usuarios registrados en Auth deben disponer de un perfil asociado en la tabla profiles, gestionado automáticamente mediante el trigger handle_new_user.

10.4. Métodos de verificación

Para verificar que el software cumple con los estándares de calidad establecidos, se han utilizado los siguientes métodos:

1. Análisis estático de código: Uso de las herramientas de inspección de Android Studio para detectar posibles fugas de memoria, optimizar el rendimiento de Jetpack Compose y corregir advertencias del compilador Kotlin.

2. Depuración mediante Logcat: Verificación constante de las trazas de red y de la inicialización del cliente Supabase mediante etiquetas personalizadas para asegurar el correcto funcionamiento de la autenticación y las API Keys.
3. Pruebas de regresión manuales: Tras cada modificación en los repositorios de datos (AuthRepository, TournamentRepository), se repiten las pruebas definidas en el apartado 10.2 para garantizar que no se introduzcan errores en funcionalidades previamente estables.
4. Simulación de latencia: Realización de pruebas en el emulador limitando la velocidad de red para verificar que la aplicación gestiona correctamente:
 - a. Timeouts
 - b. Estados de carga
 - c. Indicadores visuales como CircularProgressIndicator.

11. Distribución

11.1. Tecnología de distribución

La distribución de EasyTournament se realiza mediante un archivo APK generado desde Android Studio, permitiendo su instalación directa en dispositivos Android compatibles.

Las tecnologías utilizadas para la distribución son las siguientes:

1. Android Package (APK): La aplicación se compila en formato APK firmado, permitiendo su instalación manual en dispositivos Android sin necesidad de publicar inicialmente en Google Play Store.
2. Android Studio: La generación del APK se realiza mediante las herramientas de compilación integradas en Android Studio utilizando Gradle como sistema de automatización de builds.
3. GitHub: El repositorio remoto se utiliza como plataforma de almacenamiento y distribución del código fuente del proyecto, facilitando el acceso para evaluación y mantenimiento.

11.2. Descripción del proceso

El proceso de distribución de la aplicación se realiza siguiendo las siguientes fases:

1. Compilación del proyecto: Se ejecuta una compilación Release desde Android Studio para generar una versión optimizada y estable de la aplicación.
2. Generación del APK: Android Studio genera automáticamente el archivo APK firmado mediante Gradle, preparado para su instalación en dispositivos físicos.

3. Verificación de funcionamiento: Antes de distribuir la aplicación, se realizan pruebas finales en emuladores y dispositivos reales para comprobar:
 - a. Inicio correcto de sesión
 - b. Navegación entre pantallas
 - c. Funcionamiento de las inscripciones
 - d. Comunicación con Supabase
4. Distribución al usuario: El archivo APK puede compartirse mediante:
 - a. GitHub (la principal)
 - b. Google Drive.
 - c. Memoria USB.
 - d. Plataformas de almacenamiento en la nube.
 - e. Posibilidad de subirse a plataformas de distribución.
5. Instalación: El usuario debe habilitar la instalación desde orígenes desconocidos en Android e instalar manualmente el archivo APK en su dispositivo.

12. Manuales

Este capítulo constituye la guía de referencia para el despliegue y manejo de la aplicación EasyTournament.

12.1. Manual de instalación

La aplicación se distribuye como un proyecto de Android Studio desarrollado en Kotlin. Para su instalación, se contemplan dos métodos:

1. Instalación mediante Android Studio:
 - a. Descarga:
Clonar o descargar el repositorio del proyecto desde GitHub.
 - b. Apertura: Iniciar Android Studio y seleccionar la opción “Open”, localizando la carpeta raíz del proyecto.
 - c. Sincronización: Esperar a que Gradle descargue las dependencias necesarias:
 - i. Supabase
 - ii. Ktor
 - iii. Jetpack Compose
 - d. Ejecución: Conectar un dispositivo físico mediante USB o iniciar un emulador (mínimo API 36). Posteriormente, pulsar el botón “Run”.
2. Instalación mediante archivo APK:
 - a. Transferir el archivo apk situado en el repositorio de GitHub al dispositivo móvil.
 - b. Habilitar la instalación de aplicaciones desde “Orígenes desconocidos” en los ajustes de seguridad de Android.

- c. Ejecutar el archivo APK y completar el proceso de instalación.

12.2. Manual de uso (Guía del usuario)

La aplicación EasyTournament presenta una interfaz basada en Material 3. A continuación, se describen sus funciones principales:

1. Gestión de acceso (Autenticación): Al abrir la aplicación por primera vez, el usuario puede crear una cuenta proporcionando:
 - a. Nombre de usuario (mínimo 5 caracteres)
 - b. Correo electrónico válido
 - c. Contraseña (mínimo 6 caracteres)
 - d. Aceptación de términos legales (RGPD)
 - i. Inicio de sesión: Los usuarios registrados pueden acceder utilizando sus credenciales. La sesión permanece iniciada automáticamente para futuros accesos.
2. Pantalla principal (Home): Al acceder a la aplicación, se muestra una lista de torneos disponibles. También habrá un botón para crear un torneo. Cada torneo presenta:
 - a. Título
 - b. Juego
 - c. Organizador
 - d. Número actual de participantes
 - e. Fecha
 - f. Categoría
 - g. Modalidad
 - h. Estatus:
 - i. Si un torneo alcanza el límite máximo de participantes, el sistema lo marcará automáticamente como “Cerrado”.
 - ii. Si un torneo pasa la fecha del torneo, el sistema lo marcará automáticamente como “Finalizado”.
 - iii. Hasta entonces estará en estado “abierto”.
3. Inscripción en torneos: Al pulsar sobre un torneo, el usuario accede a la pantalla de detalles. Desde esta vista es posible:
 - a. Consultar información ampliada
 - b. Inscribirse en el torneo mediante el botón “Inscribirse” validándose en tiempo real la disponibilidad de plazas y actualizando automáticamente el contador de participantes en Supabase.
4. Perfil de usuario: Desde la sección de perfil, el usuario puede visualizar su:
 - a. Nombre de usuario
 - b. Reputación acumulada
 - c. Usuarios vinculados de plataformas externas como Steam o Riot Games
5. Gestión de usuarios (Vista de administrador): Los usuarios con rol de administrador disponen de herramientas adicionales para:

- a. Visualizar perfiles registrados ordenados alfabéticamente. (Los de rol Admin estarán abajo del todo)
 - b. Supervisar y cambiar la reputación de usuarios (aumentando o disminuyendo puntuación en 0 '5).
- 6. Configuración y ajustes: La aplicación permite personalizar determinados aspectos de la experiencia de usuario, como:
 - a. Preferencias visuales
 - b. Configuración de tema claro u oscuro
 - c. Ajustes generales de la aplicación

13. Conclusiones

13.1. Informe final

El desarrollo de EasyTournament ha permitido demostrar la viabilidad de crear una plataforma robusta de gestión de torneos utilizando tecnologías modernas de desarrollo nativo en Android.

Se ha conseguido una integración eficiente entre Jetpack Compose para la interfaz de usuario y Supabase como Backend-as-a-Service, permitiendo una comunicación fluida entre el frontend y la base de datos en tiempo real.

A nivel técnico, destaca la implementación de una arquitectura donde el estado de la aplicación se sincroniza constantemente con la base de datos. Esto permite que los usuarios visualicen de manera inmediata la disponibilidad de plazas, cambios en los torneos y actualizaciones de la comunidad.

El uso de Kotlin Coroutines ha permitido optimizar las operaciones asíncronas, evitando bloqueos en la interfaz durante las peticiones de red y mejorando notablemente la experiencia de usuario.

13.2. Resultados esperados

Los objetivos planteados al inicio del proyecto se han cumplido de forma satisfactoria:

1. Gestión de identidad: Se ha implementado un sistema de autenticación seguro compatible con los requisitos del RGPD.
2. Comunidad controlada: La plataforma dispone de un sistema de reputación que permite mantener una comunidad organizada y moderada.
3. Administración centralizada: Los administradores cuentan con herramientas específicas para supervisar perfiles, gestionar usuarios y moderar la plataforma.
4. Escalabilidad: Gracias al uso de PostgreSQL, Supabase y políticas RLS (Row Level Security), la aplicación está preparada para soportar un crecimiento progresivo de usuarios sin comprometer la seguridad ni la integridad de los datos.

13.3. Viabilidad del proyecto

El proyecto presenta una alta viabilidad tanto técnica como económica:

1. Eficiencia de costes: El uso de Supabase en su modalidad gratuita permite reducir considerablemente los costes de infraestructura durante la fase inicial del proyecto.
2. Mantenibilidad: La aplicación ha sido desarrollada siguiendo una estructura modular y organizada, facilitando futuras ampliaciones, corrección de errores y mantenimiento por parte de otros desarrolladores.
3. Rendimiento: Las pruebas realizadas demuestran que la aplicación responde correctamente incluso en condiciones de red inestables, manteniendo una experiencia de usuario fluida.

13.4. Mejoras futuras

Durante el desarrollo se han identificado posibles funcionalidades que podrían implementarse en futuras versiones:

1. Chat en tiempo real: Incorporar un sistema de mensajería para la comunicación entre participantes de un torneo.
2. Sistema de notificaciones push: Permitir avisos automáticos sobre nuevos torneos, cambios en inscripciones o eventos importantes.
3. Pasarela de pagos: Integración con plataformas como Google Pay para gestionar torneos de pago o premios monetarios.
4. Generación automática de brackets: Implementar un sistema visual de eliminatorias que permita seguir el progreso del torneo desde la aplicación.
5. Sistema de Logros y Medallas: Implementar un sistema de recompensas visuales (badges) en el perfil del usuario para premiar hitos como "Ganar 5 torneos", "Usuario ejemplar" (reputación máxima) o "Organizador veterano".
6. Historial de Enfrentamientos: Una sección donde los usuarios puedan ver sus estadísticas personales contra otros jugadores específicos, fomentando la competitividad sana.
7. Integración con Discord/Twitch: Conectar las cuentas de usuario con sus perfiles de streaming para mostrar si un torneo está siendo retransmitido en directo directamente desde la ficha de la App.
8. Muro de Noticias y Comunidad: Un muro de noticias general donde los administradores puedan publicar anuncios importantes, cambios de reglas o fotos de los eventos finalizados.
9. Sistema de Reportes: Los usuarios podrán enviar tickets a la administración para reportar usuarios o resolver problemas o dudas.
10. Más ajustes: Se implementarán cambios de idiomas y notificaciones. Además de más posibilidades para personalizar la experiencia de los usuarios.
11. Uso de avatar_url para usar fotos de perfil.

14. Anexos

14.1 Registro de Pruebas

PR-01 (Autenticación)

Paso 1: Se accede a la ventana de registro

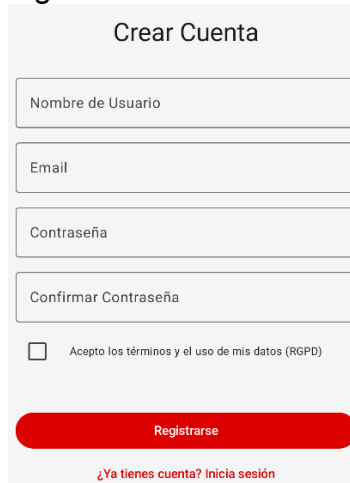
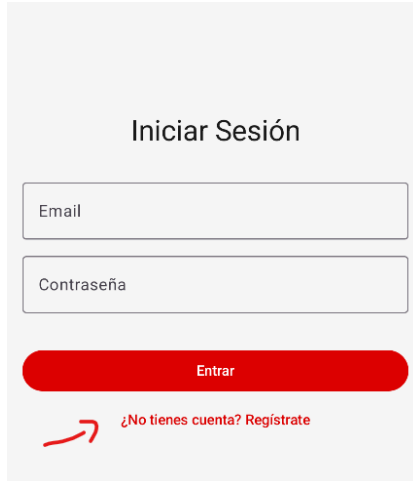


Imagen 19 Prueba de registro

Imagen 20 Ventana de registro

Paso 2: Se insertan datos, se aceptan los términos y se da a registrarse.

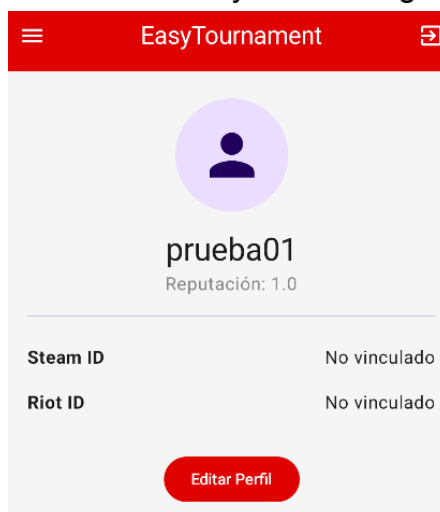
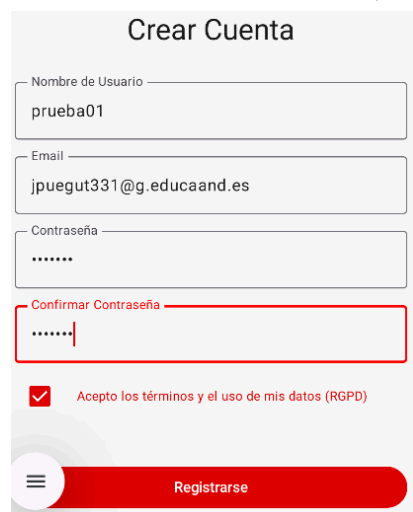


Imagen 21 Datos de Registro

Imagen 22 Perfil de Usuario Registrado

PR-02 (Validación de Autenticación)

Paso 1: Se accede a la ventana de registro. Y se inserta un valor de email que ya esté registrado.

Crear Cuenta

Nombre de Usuario
prueba2

Email
jpuegut331@g.educaand.es

Contraseña
.....

Confirmar Contraseña
.....

☐ Acepto los términos y el uso de mis datos (RGPD)

Registrarse

[¿Ya tienes cuenta? Inicia sesión](#)

Imagen 23 Prueba de registro con usuario registrado
Paso 2: Darle a registrarse. Se mostrará un toast impidiendo el registro.

Crear Cuenta

Nombre de Usuario
prueba2

Email
jpuegut331@g.educaand.es

Contraseña
.....

Confirmar Contraseña
.....

☒ Acepto los términos y el uso de mis datos (RGPD)

Registrarse

[¿Ya tienes cuenta? Inicia sesión](#)

Este email ya está en uso

Imagen 24 Fallo de la prueba

PR-03 (Inscripción de torneo)

Se inicia sesión y en la ventana de explorar se elige el torneo al que uno se quiere inscribir.

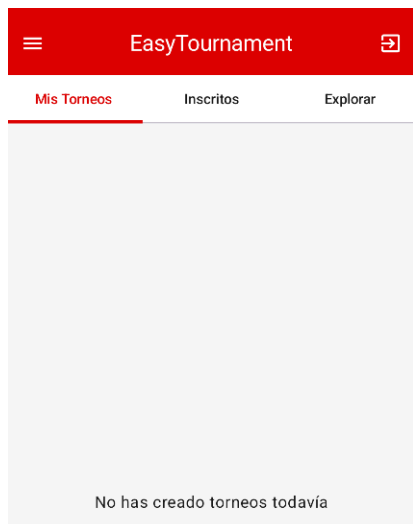


Imagen 25 Visionado de torneos

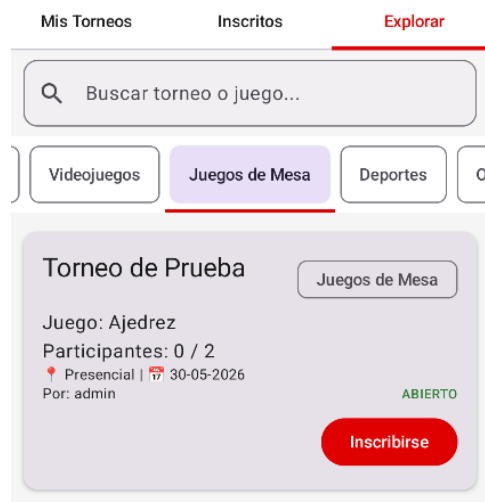


Imagen 26 Card de Torneo

Paso 2: Se le da al botón de Inscribirse. Este torneo aparecerá en la ventana de Inscritos.

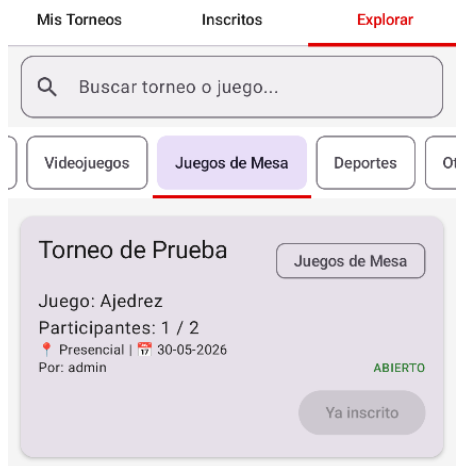


Imagen 27 Tras presionar botón de inscripción

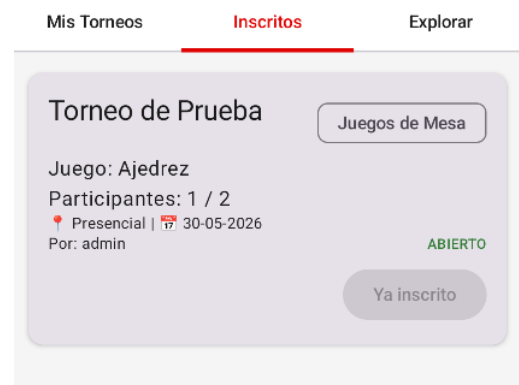


Imagen 28 Ventana de Inscritos

PR-04 (Concurrencia)

Paso 1: Tras registrar otro usuario al torneo anterior iniciamos sesión se inicia sesión con un tercer usuario para acceder al torneo anterior 2 dos personas. No se podrá acceder al mismo ya que el botón de Inscribirse estará bloqueado.

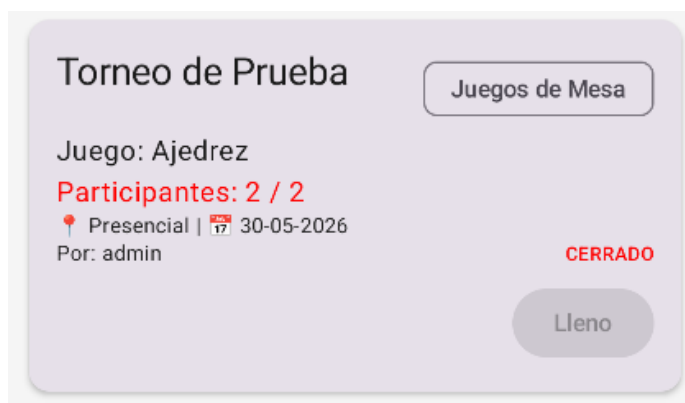


Imagen 29 Torneo completo

PR-05: Seguridad RLS

Paso 1: Para borrar un torneo creado por otra persona hacemos click en el Card del Torneo a borrar.



Imagen 30 Prueba de borrado

Paso 2: Como el usuario no es el creador o admin no aparece el botón de borrar.

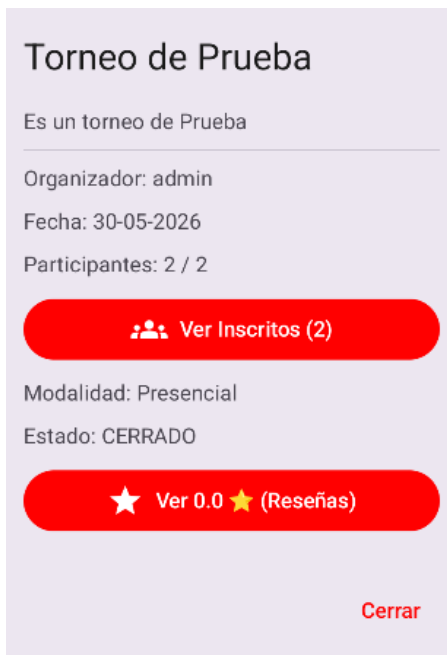


Imagen 31 Borrado imposible

PR-06 (Reputación)

Paso 1: Se dirige al HomeScreen con el usuario con reputación negativa. Se comprueba que no aparece el botón para añadir torneos abajo a la derecha.

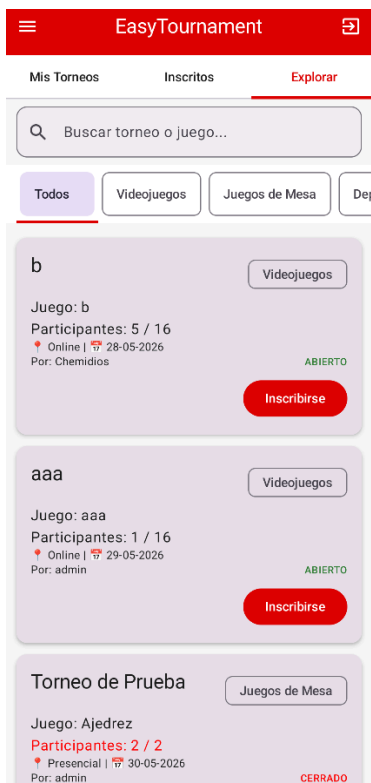


Imagen 32 Imposibilidad de añadir torneo

14.2 Índice de imágenes y tablas

Índice de imágenes

Imagen 1. Iniciar Sesión Modo Claro.....	27
Imagen 2. Registrar Modo Claro.....	27
Imagen 3. Iniciar Sesión Modo Oscuro.....	27
Imagen 4. Registrar Modo Oscuro.....	27
Imagen 5. HomeScreen Modo Claro.....	28
Imagen 6. Registrar Torneo Modo Claro.....	28
Imagen 7. HomeScreen Modo Oscuro.....	29
Imagen 8. Registrar Torneo Modo Oscuro.....	29
Imagen 9. SettingsScreen Modo Claro.....	30
Imagen 10. SettingsScreen Modo Oscuro.....	30
Imagen 11 ProfileScreen Modo Claro.....	31
Imagen 12 ProfileScreen Modo Oscuro.....	31
Imagen 13 UsersScreen Modo Claro.....	32
Imagen 14 UsersScreen Modo Oscuro.....	32
Imagen 15 ModalNavigationDrawer Modo Claro.....	33
Imagen 16 ModalNavigationDrawer Modo Oscuro.....	33
Imagen 17 Diagrama de Clases.....	35
Imagen 18 Diagrama de Casos de Uso.....	36
Imagen 19 Prueba de registro.....	45
Imagen 20 Ventana de registro.....	45
Imagen 21 Datos de Registro.....	45
Imagen 22 Perfil de Usuario Registrado.....	45
Imagen 23 Prueba de registro con usuario registrado.....	46
Imagen 24 Fallo de la prueba.....	46
Imagen 25 Visionado de torneos.....	47
Imagen 26 Card de Torneo.....	47
Imagen 27 Tras presionar botón de inscripción.....	47
Imagen 28 Ventana de Inscritos.....	47
Imagen 29 Torneo completo.....	48
Imagen 30 Prueba de Borrado.....	48
Imagen 31 Borrado imposible.....	49
Imagen 32 Imposibilidad de añadir torneo.....	49

Índice de tablas

Tabla 1 Tabla de Riesgos.....	13
Tabla 2 Pruebas de aplicación.....	39

15. Bibliografía

[Análisis comparativo de aplicaciones similares](#)
[Corrutinas de Kotlin en Android | Android Developers](#)
[Serialization | Kotlin Documentation](#)
[Compose Material 3 | Jetpack | Android Developers](#)
[PostgreSQL: Documentation: 18: CREATE TRIGGER](#)
[Row Level Security | Supabase Docs](#)